

1. Emanuel — Bienvenida, Introducción al Sistema y Frontend

*"¡Hola a todos Buenos Días! El día de hoy les vamos a presentar nuestro proyecto integrador: una **Mascota Virtual estilo Tamagotchi We.***

El objetivo de esta presentación es mostrar cómo aplicamos los conocimientos adquiridos durante el cuatrimestre para desarrollar una aplicación web funcional, utilizando Python como lenguaje principal del servidor y HTML, CSS y JavaScript para la interfaz del usuario.

*Para entender nuestro código de forma sencilla, lo dividimos en **Frontend y Backend.***

*El **Frontend** es la cara del sistema: **todo lo que el usuario ve y toca en pantalla.** Si ven la estructura de nuestro proyecto en la carpeta templates, ahí están las pantallas HTML como tienda.html para comprar insumos, veterinario.html para la salud, inventario.html para ver los objetos guardados, y nuestros minijuegos: gato.html, palabras.html y aventura.html para la serpiente.*

El usuario interactúa con estas vistas, pero para que el progreso no se borre al cerrar la página, necesitamos la lógica de Python en el Backend, que les explicará Lupe."

2. Lupe — Backend en Python, Flask y Base de Datos

*"¡Gracias, Emanuel! Como él dijo, si el Frontend es la cara, el **Backend es el cerebro que no se ve,** y aquí **Python es el encargado de procesar toda la lógica del sistema.***

*Usamos **Flask**, que es un microframework de Python. Lo elegimos porque es sumamente ligero y nos permite procesar la lógica de negocio en nuestro archivo principal main.py de forma limpia.*

*Ahí controlamos el inicio de sesión y comunicamos el sistema con la **Base de Datos MySQL (tamagotchi.sql).** Para lograr esta conexión creamos el script conexion.py, usando el conector nativo de Python.*

Un detalle clave es que aprovechamos la potencia de las consultas SQL enviadas desde Python usando funciones como LEAST y GREATEST. Esto asegura que atributos como el hambre o la salud nunca bajen de 0 ni pasen de 100" permitiendo mantener los valores dentro de límites válidos, evitando errores en la información almacenada.

3. Pau — Comunicación HTTP, JSON, Canvas y Conclusión

"Ahora, ¿cómo se comunican las pantallas HTML con el servidor en Python? Lo hacen mediante peticiones HTTP.

Con el método GET obtenemos los datos de la mascota desde MySQL para mostrarlos en la pantalla. En cambio, cuando el usuario realiza una acción, como curar a la mascota, comprar en la tienda o ganar puntos, utilizamos el método POST para enviar la información al servidor en formato JSON.

El JSON es un formato de texto ligero que utilizamos para intercambiar información entre el navegador y el servidor mediante `fetch()` de JavaScript. Gracias a esto, los cambios se guardan en MySQL y la interfaz se actualiza de forma inmediata, sin necesidad de recargar la página.

Por último, para el juego de Snake en `aventura.html` utilizamos HTML5 Canvas 2D, controlado con `requestAnimationFrame()`, lo que permite mantener una animación fluida de aproximadamente 60 FPS.

Como conclusión, este proyecto nos demostró el poder de Python no solo como un lenguaje de consola o de scripting, sino como el motor principal para desarrollar aplicaciones web dinámicas y persistentes, integrando una interfaz interactiva, una base de datos y comunicación en tiempo real entre el cliente y el servidor.

¡Muchas gracias por su atención! Quedamos abiertos a sus preguntas."

Aquí tienes el **acordeón definitivo (Cheat Sheet completo)** para la defensa de tu proyecto. Está dividido por categorías clave para que puedas repasarlo rápido antes de pasar a exponer.

1. Conceptos Base y Arquitectura

Pregunta	Respuesta Directa y Al Punto
¿Qué es Frontend?	Es la cara del sistema, lo que el usuario ve e interactúa . En nuestro proyecto son todas las plantillas HTML dentro de la carpeta <code>templates/</code> (<code>tienda.html</code> , <code>veterinario.html</code> , <code>minijuegos</code>).

Pregunta	Respuesta Directa y Al Punto
¿Qué es Backend?	Es el cerebro que no se ve . Es la lógica en Python (main.py y Flask) que procesa las peticiones, valida reglas del juego y decide qué datos guardar.
¿Qué es la Base de Datos?	Es la memoria permanente (tamagotchi.sql). Mantiene el estado de la mascota, monedas e inventario guardados aunque cierres el navegador.
¿Por qué usaron Flask y no Django?	Porque Flask es un microframework ligero y flexible . Nos da control total para escribir las rutas en Python sin la sobrecarga de un framework gigante.
¿Qué hace conexion.py?	Aplica el principio de modularidad . Separa la función de conexión a MySQL de la lógica principal (main.py) para no repetir código.
¿Qué hace iniciar_juego.bat?	Es un archivo ejecutable de Windows que activa el entorno virtual de Python y arranca el servidor de Flask en un solo clic.

2. Comunicación HTTP, JSON y Sesiones

Pregunta	Respuesta Directa y Al Punto
¿Diferencia entre GET y POST?	GET pide datos al servidor para cargar/ver una página. POST envía datos al servidor para hacer cambios (comprar, curar, guardar puntaje).
¿Qué es JSON y por qué lo usan?	Es un formato de texto ligero para intercambiar datos. Lo usamos con fetch() en JS para guardar cosas en segundo plano sin recargar la página .
¿Qué es Jinja2?	Es el motor de plantillas de Flask. Permite inyectar variables de Python directo dentro de nuestros archivos HTML.

Pregunta	Respuesta Directa y Al Punto
¿Cómo funcionan las sesiones (session)?	Guardan un ID del usuario en el servidor mientras navega. Si alguien intenta entrar a tienda.html sin haber iniciado sesión, Flask lo detecta y lo redirige al login.

3. Frontend y Minijuegos (DOM vs Canvas)

Pregunta	Respuesta Directa y Al Punto
¿Cómo funcionan el Gato y Palabras?	Manipulan el DOM (los elementos HTML directamente) con JavaScript vanilla para registrar clics, letras y victorias.
¿Qué es HTML5 Canvas 2D?	Es una etiqueta HTML que funciona como un lienzo en blanco donde dibujamos mediante JavaScript los pixeles del juego de Snake en tiempo real.
¿Por qué usaron requestAnimationFrame en Snake y no setInterval?	Porque setInterval causa tirones. requestAnimationFrame sincroniza los movimientos a los 60 FPS del navegador y se pausa si cambias de pestaña.

4. Base de Datos y Consultas SQL

Pregunta	Respuesta Directa y Al Punto
¿Cómo garantizan que la salud/hambre no pasen de 100 ni bajen de 0?	Usamos las funciones LEAST() y GREATEST() en la misma consulta UPDATE de SQL. La base de datos impone el límite de forma automática e inmune a errores.
¿Cómo se procesa la compra en la Tienda?	Se manda un POST con el ID del producto -> Flask verifica en MySQL si hay monedas suficientes -> Si

Pregunta	Respuesta Directa y Al Punto
	alcanza, resta monedas e inserta el ítem en la tabla inventario.

5. Código de Python Específico

Pregunta	Respuesta Directa y Al Punto
¿Qué hace <code>if __name__ == "__main__":</code> ?	Verifica si el archivo se ejecutó directamente para iniciar el servidor. Si el archivo fuera importado desde otro script, la condición evita que el servidor arranque por error.
¿Para qué sirve <code>debug=True</code> ?	Reinicia el servidor automáticamente cuando guardas cambios en el código y muestra errores detallados en pantalla. Solo se usa en desarrollo.

Tip de oro para la presentación:

Si les preguntan algo técnico que duden en responder, recuerden la frase clave:

*"Todo lo manejamos conectando la lógica de **Python** con peticiones en segundo plano en **JSON**, para que la base de datos **MySQL** siempre mantenga la información actualizada sin afectar la experiencia del usuario."*